🎮 Control Statements

Mastering Decision Making in C Programming

🔀 Sequence 🔀 Selection 🔁 Repetition

📋 Overview of Control Statements

🎯 Control statements determine the **"flow of control"** in a program and specify the order in which instructions are executed.

🔢 Sequence

Executing one instruction after another in order

🔀 Selection

Choosing between different sections of code

🔁 Repetition

Executing the same section of code repeatedly

C Provides Three Selection Structures:

if	if...else	switch
Simple conditional	Two-way branching	Multi-way branching

🤔 Decision Control Statements

🔀 In a C program, a decision causes a **one-time jump** to a different part of the program, depending on the value of an expression.

Types of Decision Control Statements:

- **if...else statement**
  Chooses between two alternatives
- **switch statement**
  Creates branches for multiple alternatives

💡 **Forms of if-statement:** Simple if, If-else, Nested if-else, Else if

The if Statement

✅ Used to execute an instruction or sequence/block of instructions **only if a condition is fulfilled**.

Syntax:

```
if (condition)
    statement;
```

```
if (condition)
{
    block of statements;
}
```

🔄 **How it works:** The expression is evaluated first. If the condition is true, the statement is executed. If false, the statement is ignored and the program continues with the next instruction.

💻 if Statement Example

Calculate Net Salary

Calculate the net salary of an employee, if a tax of 15% is levied on his gross-salary if it exceeds Rs. 10,000/- per month.

```
/*Program to calculate the net salary of an employee */
#include<stdio.h>
main() {
    float gross_salary, net_salary;

    printf("Enter gross salary of an employee\n");
    scanf("%f", &gross_salary);

    if(gross_salary < 10000)
        net_salary = gross_salary;

    net_salary = gross_salary - 0.15 * gross_salary;
    printf("\nNet salary is Rs.%.2f\n", net_salary);
}
```

⚠️ **Note:** This program has a logical error - the tax is always deducted regardless of salary amount!

The if-else Statement

Used when a **different sequence of instructions** is to be executed depending on the logical value (True/False) of the condition evaluated.

Syntax:

```
if (condition)
    Statement_1;
else
    Statement_2;
Statement_3;
```

```
if (condition) {
    Statements_1_Block;
} else {
    Statements_2_Block;
}
Statements_3_Block;
```

🔄 **How it works:** If the condition is true, Statements_1_Block executes; otherwise Statements_2_Block will get executed. In both cases, control is then transferred to Statements_3.

💻 if-else Statement Example

```
#include<stdio.h>
#include<conio.h>
void main() {
    int no;
    clrscr();

    printf("\n Enter Number :");
    scanf("%d", &no);

    if(no > 0) {
        printf("\n\n Number is greater than 0 !");
    } else {
        if(no == 0) {
            printf("\n\n It is 0 !");
        } else {
            printf("Number is less than 0 !");
        }
    }
```

```
    getch();
}
```

Output:

```
Enter Number: 0
 It is 0!
```

🎛️ The switch Statement

🔀 A **multiple or multiway branching** decision making statement that checks the value of an expression against case values.

Syntax:

```
switch(expression) {
    case expr1:
        statements;
        break;
    case expr2:
        statements;
        break;
    ---------------------------
```

```
    case exprn:
        statements;
        break;
    default:
        statements;
}
```

Key Components:

- switch, case, break are keywords
- expr1, expr2 are 'case labels'
- break causes exit from switch
- default case is optional

Why Use switch?

- Overcomes complexity of nested if-else
- Makes program easier to read
- Simplifies multiple condition checks

💻 switch Statement Example

```
#include<stdio.h>
#include<conio.h>
void main() {
    int no;
    clrscr();

    printf("\n Enter any number from 1 to 3 :");
    scanf("%d", &no);

    switch(no) {
```

```
        case 1:
            printf("\n\n It is 1 !");
            break;
        case 2:
            printf("\n\n It is 2 !");
            break;
        case 3:
            printf("\n\n It is 3 !");
            break;
        default:
            printf("\n\n Invalid number !");
    }

    getch();
}
```

Output:

```
Enter any number from 1 to 3 : 3
It is 3 !
```

📋 Rules and Advantages of switch case

📝 Rules for declaring switch case:

- The case label should be integer or character constant.
- Each compound statement of a switch case should contain break statement to exit from case.
- Case labels must end with (:) colon.

✅ Advantages of switch case:

🎯

Easy to use

Simpler syntax than nested if-else

🔍

Easy to find errors

Clear structure helps identify issues

🛠️

Debugging is made easy

Simpler to trace execution flow

📉

Complexity minimized

Reduces program complexity